Assignment Kit for Coding Standard

Personal Software Process for Engineers: Part I

Personal Software Process for Engineers: Part I

Assignment Kit for the Coding Standard

Overview

Overview

This assignment kit covers the following topics.

Prerequisites

Prerequisites

- Read Chapter 4
- Complete Size Counting Standard

Objectives

The objectives of the coding standard are to

- establish a consistent set of coding practices
- provide criteria for judging the quality of the code that you produce
- facilitate size counting by ensuring your programs are written so they can be readily counted
- for LOC counting, require that there be a separate physical line for each logical line of code

Coding standard requirements

Coding standard requirements	Produce, document, and submit a completed coding standard that calls for quality coding practices. For LOC counting, ensure that a separate physical source line is used for each logical line of code. Submit the coding standard with your program 2 assignment package.

Example coding Standard

Coding standard example

Pages 5 and 6 of this workbook contain an example C++ coding standard.

Notes about the example
- Since it is an example, tailor it to meet your personal needs.
- If you have an existing organizational standard, consider using it for the PSP exercises.

Continued on next page

Example C++ Coding Standard

Purpose	To guide implementation of C++ programs
Program Headers	Begin all programs with a descriptive header.
Header Format	/**/ /* Program Assignment: the program number */ /* Name: your name */ /* Date: the date you started developing the program */ /* Description: a short description of the program and what it does */ /**/
Listing Contents	Provide a summary of the listing contents
Contents Example	/**/ /* Listing Contents: */ /* Reuse instructions */ /* Modification instructions */ /* Compilation instructions */ /* Includes */ /* Class declarations: */ /* CData */ /* ASet */ /* Source code in c:/classes/CData.cpp: */ /* CData */ /* CData() */ /* Empty() */ /**/

(continued)

Example C++ Coding Standard (continued)

Reuse Instructions	- Describe how the program is used: declaration format, parameter values, types, and formats. - Provide warnings of illegal values, overflow conditions, or other conditions that could potentially result in improper operation.
Reuse Instruction Example	/**/ /* Reuse instructions */ /* int PrintLine(char *line_of_character) */ /* Purpose: to print string, ‘line_of_character’, on one print line */ /* Limitations: the line length must not exceed LINE_LENGTH */ /* Return 0 if printer not ready to print, else 1 */ /**/
Identifiers	Use descriptive names for all variable, function names, constants, and other identifiers. Avoid abbreviations or single-letter variables.
Identifier Example	Int number_of_students; /* This is GOOD */ Float: x4, j, ftave; /* This is BAD */
Comments	- Document the code so the reader can understand its operation. - Comments should explain both the purpose and behavior of the code. - Comment variable declarations to indicate their purpose.
Good Comment	If(record_count > limit) /* have all records been processed? */
Bad Comment	If(record_count > limit) /* check if record count exceeds limit */
Major Sections	Precede major program sections by a block comment that describes the processing done in the next section.
Example	/**/ /* The program section examines the contents of the array ‘grades’ and calcu- */ /* lates the average class grade. */ /**/
Blank Spaces	- Write programs with sufficient spacing so they do not appear crowded. - Separate every program construct with at least one space.
Indenting	- Indent each brace level from the preceding level. - Open and close braces should be on lines by themselves and aligned.

Indenting Example	while (miss_distance > threshold)
	{
	  success_code = move_robot (target _location);
	  if (success_code == MOVE_FAILED)
	  {
	    printf(“The robot move has failed.\n”);
	  }
	}

Capitalization	- Capitalize all defines. - Lowercase all other identifiers and reserved words. - To make them readable, user messages may use mixed case.
Capitalization Examples	#define DEFAULT-NUMBER-OF-STUDENTS 15 int class-size = DEFAULT-NUMBER-OF-STUDENTS;

Evaluation criteria and suggestions

Evaluation criteria	Your standard must be • complete • legible
Suggestions	Keep your standards simple and short. Do not hesitate to copy or build on the PSP materials.

Coding Standard Template

Purpose	Guiar el desarrollo de programas en Java para el análisis de regresión lineal
Program Headers	Program Headers: Comenzar todos los programas con un header descriptivo
Header Format	/** * Program Assignment: PSP 3A * Name: [Karla Sofía Castro Pérez] * Date: [25-11-2025] * Description: Breve descripción de lo que hace la clase */
Listing Contents	Listing Contents: Proporcionar un resumen del contenido del listado
Contents Example	/** * Listing Contents: * - Declaración de clase Output * - Método writeData para escribir archivos * - Manejo de excepciones con try-with-resources */
Reuse Instructions	• Input.java: String readData(String inFile) - Lee un archivo y retorna su contenido como String • Output.java: void writeData(String outFile, String outText) - Escribe texto en un archivo • EstimacionCorLineal.java: Contiene métodos para cálculos estadísticos (sumX, sumY, getB1, getRXY, etc.) • Limitaciones: Los archivos deben existir y ser accesibles, los datos deben estar en formato numérico separado por espacios

Reuse Example	// Para usar la clase Input:
	Input reader = new Input();
	String contenido = reader.readData("datos.txt");
	
	// Para usar la clase Output:
	Output writer = new Output();
	writer.writeData("resultados.txt", "Texto a guardar");

Identifiers	Identifiers: Usar nombres descriptivos para todas las variables, nombres de funciones, constantes y otros identificadores. Evitar abreviaturas o variables de una sola letra.

Identifier Example	// BUENO:
	double sumaDeCalificaciones;
	String nombreDelArchivo;
	int numeroDeEstudiantes;
	
	// MALO:
	double s;
	String n;
	int x;

(continued)

Coding Standard Template (continued)

Comments	• Documentar el código para que el lector pueda entender su operación • Los comentarios deben explicar tanto el propósito como el comportamiento del código • Comentar declaraciones de variables para indicar su propósito

Good Comment	// Calcular el promedio de los valores de X
	for (double valor : datosX) {
	    suma += valor;

Bad Comment	// Hacer un loop
	for (double valor : datosX) {
	    suma += valor;
	}

Major Sections	Precede major program sections by a block comment that describes the processing that is done in the next section
Example	
Blank Spaces	• Escribir programas con suficiente espaciado para que no parezcan amontonados • Separar cada construcción del programa con al menos un espacio
Indenting	• Indentar cada nivel de llave respecto al anterior • Las llaves de apertura y cierre deben estar en líneas por sí mismas y alineadas

Indenting Example	public void sumX(double[] datosX) {
	    double suma = 0;
	    for (double valor : datosX) {
	        suma += valor;
	    }
	    this.dblSumX = suma;
	}

Capitalization	• Capitalizar todos los defines (constantes) • Minúsculas para otros identificadores y palabras reservadas • Los mensajes de salida al usuario pueden usar mayúsculas y minúsculas para una presentación limpia

Capitalization Example	// Constantes en MAYÚSCULAS:
	private static final int MAXIMO_DATOS = 100;
	
	// Variables y métodos en camelCase:
	private double promedioEstudiantes;
	public void calcularPromedio() {}
	
	// Mensajes al usuario con formato legible:
	System.out.println("Ingrese el nombre del archivo a analizar:");

